

William Anderson

Senior Software Engineer

Seminole, FL | williamandresondev@outlook.com | +1 (407) 801-1287 |
<https://www.linkedin.com/in/william-andreson-39b88b3aa>

Professional Summary

Senior Rust Software Engineer with experience across fintech, cloud networking, SaaS, and developer-platform industries, building high-performance services with **Rust 1.56–1.85**, **Tokio 1.x**, **Axum 0.6–0.7**, **Actix Web 4.x**, **PostgreSQL 13–16**, **Kafka 2.8–3.x**, and **AWS**. Specialized in memory-safe backend systems, low-latency APIs, async workers, distributed tracing, secure CI/CD, and zero-downtime migrations from legacy Node.js, Java, and C++ services into Rust-based platforms, improving reliability, reducing runtime failures, fixing concurrency issues, and supporting scalable production delivery across regulated customer-facing environments.

Technical Skills

- **Languages:** **Rust 1.56–1.85**, C++11–17, Python 3.8–3.12, TypeScript 4.x–5.x, Java 8–17, Bash, SQL
- **Rust Backend:** **Tokio 1.x**, **Axum 0.6–0.7**, **Actix Web 4.x**, Tower, Serde, SQLx, Diesel, Reqwest, Rayon, Clap, Cargo, Rustfmt, Clippy
- **Architecture:** Microservices, event-driven systems, API gateways, async workers, message queues, distributed tracing, service decomposition
- **Cloud & DevOps:** AWS, ECS, EKS, Lambda, S3, RDS, CloudWatch, Docker, Kubernetes, Terraform, GitHub Actions, GitLab CI
- **Databases & Messaging:** PostgreSQL 13–16, MySQL 8.x, Redis 6.x–7.x, Kafka 2.8–3.x, RabbitMQ, DynamoDB
- **Observability & Security:** OpenTelemetry, Prometheus, Grafana, Jaeger, structured logging, OAuth2, JWT, mTLS, vulnerability scanning

Professional Experience

Viaro Networks Inc.

Senior Software Engineer | January 2021 – December 2025

- Led development of **Rust 1.56–1.85** backend services for cloud networking and customer-facing SaaS platforms, using **Tokio 1.x**, **Axum 0.6–0.7**, and **PostgreSQL 13–16** to deliver memory-safe APIs with predictable latency.
- Migrated selected Node.js and Java services into **Rust async microservices**, reducing runtime crashes by **38%** and improving request throughput by **46%** under peak production traffic.
- Implemented a high-volume event processing layer with **Kafka 3.x**, Rust consumers, retry-safe workers, and dead-letter queues, helping the platform process customer activity streams with stronger delivery guarantees.
- Fixed a production memory growth issue caused by unbounded async task spawning by adding bounded channels, backpressure controls, timeout handling, and structured cancellation across **Tokio** workers.
- Designed **Axum** REST APIs with middleware for authentication, request tracing, validation, rate limiting, and response normalization, improving service consistency across multiple backend teams.
- Migrated legacy Rust services from older edition patterns into **Rust 2021 and Rust 2024-ready code**, resolving lifetime warnings, dependency conflicts, and unsafe build assumptions before release.

- Built secure integration services using **OAuth2**, JWT validation, mTLS, and role-based authorization, reducing unauthorized access risk and improving audit readiness for enterprise clients.
- Improved SQL performance by replacing inefficient ORM-style access with **SQLx** query validation, indexed joins, pagination, and connection pool tuning, reducing slow queries by **41%**.
- Created reusable Rust libraries for error handling, logging, telemetry, configuration loading, and API clients, helping developers apply the same production standards across services.
- Resolved intermittent deployment failures caused by mismatched Cargo feature flags by separating workspace profiles, locking dependency versions, and adding CI validation with **Clippy** and tests.
- Implemented **OpenTelemetry**, Prometheus metrics, Grafana dashboards, and trace correlation IDs, reducing incident investigation time by **35%** during production support.
- Used Rust flexibly across API services, CLI tooling, background workers, and data synchronization jobs, choosing **Axum**, **Tokio**, or lightweight binaries based on workload needs.

VividCloud

Software Engineer | January 2016 – December 2020

- Developed cloud platform services with **Rust 1.31–1.48**, C++17, Python 3.x, PostgreSQL 10–12, Redis 5.x, and Docker, supporting infrastructure automation and distributed backend workloads.
- Introduced Rust for selected performance-sensitive data processing components where memory safety and predictable CPU usage were more valuable than traditional scripting approaches.
- Reworked a slow Python-based log parser into a compiled Rust command-line tool, improving large-file processing speed by **52%** while reducing operational timeout failures.
- Fixed a concurrency bug in a worker service where duplicated retry messages caused inconsistent state, adding idempotency keys, transactional updates, and safer queue acknowledgment logic.
- Built internal APIs using **Actix Web 2.x–3.x** and PostgreSQL, exposing service health, deployment metadata, and customer usage data for operational dashboards.
- Migrated several older C++ utilities into safer Rust modules by replacing manual memory handling with ownership-based design, typed errors, and automated regression tests.
- Improved deployment reliability by adding Docker multi-stage builds, GitLab CI pipelines, dependency caching, and release artifacts for Rust and Python services.
- Implemented feature flags for backend rollout control, allowing the team to release new customer settings gradually and reduce rollback risk during production changes.
- Optimized Redis caching and PostgreSQL indexing around customer lookup flows, improving dashboard load performance by **34%** across frequently accessed accounts.
- Added structured logging, request IDs, and Prometheus metrics to backend services, making recurring production issues easier to reproduce and verify after fixes.
- Collaborated across backend, DevOps, and QA teams to apply Rust where it solved reliability problems while keeping Python and C++ where they remained the better fit.

Minimalist Moon

Software Developer | January 2013 – December 2015

- Built backend services and internal tools using C++11, Node.js 0.12–4.x, Python 2.7–3.5, PostgreSQL 9.x, MySQL 5.6, Redis, and Linux shell scripting for early SaaS products.
- Developed REST endpoints, scheduled workers, and data import utilities that supported customer onboarding, reporting workflows, and operational dashboards for small business clients.
- Resolved a recurring file upload failure caused by incomplete stream handling by adding checksum validation, retry logic, and safer temporary file cleanup across Linux services.
- Improved backend response performance by tuning SQL queries, adding missing indexes, and reducing repeated database calls, lowering average report generation time by **29%**.
- Implemented new account configuration features with server-side validation, audit logging, and role-aware access rules, helping support teams manage client settings more safely.

- Maintained C++ utilities for batch processing and system integrations, applying safer pointer usage, clearer error paths, and regression tests to reduce production defects.
- Migrated older callback-heavy Node.js modules into cleaner service layers with promise-based control flow where appropriate, improving readability and reducing async handling mistakes.
- Built deployment scripts, cron-based workers, and environment configuration tools that helped engineers ship backend changes consistently across staging and production servers.
- Fixed timezone-related reporting bugs by normalizing stored timestamps, validating client locale rules, and adding test coverage for month-end and daylight-saving scenarios.
- Worked flexibly across backend code, database tuning, Linux operations, and customer issue debugging, building a strong foundation for later systems-level Rust engineering.

Microsoft

Software Developer Intern | *July 2012 – December 2012*

- Supported internal engineering tools using C#, .NET Framework 4.x, C++11, PowerShell, SQL Server, and Windows services, focusing on reliability improvements and developer productivity.
- Built small automation utilities that reduced repetitive validation work for senior engineers and helped teams verify build outputs more consistently before internal release.
- Investigated a build pipeline error caused by inconsistent environment variables, documenting the root cause and adding validation scripts that made failures easier to diagnose.
- Enhanced internal dashboard components by improving data formatting, exception handling, and query filtering, helping engineers review system status with fewer manual checks.
- Assisted with C++ module testing by adding edge-case coverage around parsing, file handling, and error reporting, reducing repeated defects found during review cycles.
- Created PowerShell scripts for log collection and deployment verification, making it easier to reproduce environment-specific failures across developer machines.
- Reviewed production-like issues with senior engineers and learned how typed languages, compiler checks, and disciplined testing reduce defects before runtime.
- Improved SQL Server stored procedure performance by reviewing query plans, removing unnecessary joins, and validating results against existing internal reports.
- Built a strong engineering base in debugging, systems thinking, performance analysis, and secure coding practices that later transferred naturally into **Rust** backend development.

Education

Florida State University

Bachelor's Degree in Computer Science | 2008 – 2012